

Python comedy sketch, but we use these as metasyntactic variables only; it's inadvisable to use them in real-world code.

The best policy is to write your code in a way that non-native English speakers can readily understand: polite, direct, and humorless. My former co-worker might have thought `gooseDownload()` was a funny joke, but nothing kills a joke faster than having to explain it.

Don't Overwrite Built-in Names

You should also never use Python's built-in names for your own variables. For example, if you name a variable `list` or `set`, you'll overwrite Python's `list()` and `set()` functions, possibly causing bugs later in your code. The `list()` function creates list objects. But overwriting it can lead to this error:

```
>>> list(range(5))
[0, 1, 2, 3, 4]
❶ >>> list = ['cat', 'dog', 'moose']
❷ >>> list(range(5))
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'list' object is not callable
```

If we assign a list value to the name `list` ❶, we'll lose the original `list()` function. Attempting to call `list()` ❷ would result in a `TypeError`. To find out whether Python is already using a name, type it into the interactive shell or try to import it. You'll get a `NameError` or `ModuleNotFoundError` if the name isn't being used. For example, Python uses the names `open` and `test` but doesn't use `spam` and `eggs`:

```
>>> open
<built-in function open >
>>> import test
>>> spam
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'spam' is not defined
>>> import eggs
Traceback (most recent call last):
```
